Malware Detection on EMBER2024: Manual Tuning versus AutoML

How much do random-split scores overstate real-world performance, and do AutoML tools protect against, or fall for, leakage and temporal drift?

Course	AutoML Skill-Based Project, Evaluation IA-3, Part B (30 marks)
Programme	Final-year B.Tech Artificial Intelligence and Data Science, Karunya Institute of Technology and Sciences
Student	Isaac Benjamin David
Register number	URK23AI1035
Team members	TODO: team members to be added by the student
Code	https://github.com/Turbo-Soggy/automl
Report built	2026-10-30 by scripts/build_report.py (all numbers read from reports/ at build time)

Dataset: EMBER2024 Win64 PE files, 160,000 training, 40,000 random-test and 40,000 future-test files, 2,568 static features. Best model: tuned LightGBM, test_future ROC-AUC 0.9981.

Contents

Abstract 3

1. Problem statement and research question 3

2. Dataset 4

2.1 The duplicate finding . . . 4

2.2 Splits and class balance . . . 4

2.3 Feature families . . . 5

3. Methodology and workflow 5

3.1 Design decisions . . . 5

4. Step 2: baseline and hyperparameter tuning 6

5. Step 3: AutoML and tool comparison 7

5.1 Leaderboards . . . 8

5.2 Preprocessing and interpretability . . . 9

6. Step 4: critical evaluation 10

6.1 The answer in short . . . 10

6.2 Random vs temporal split . . . 11

6.3 Deliberate leakage . . . 12

6.4 Do AutoML tools protect against drift? . . . 13

6.5 Bias, imbalance and base rate . . . 14

6.6 Explainability (SHAP) . . . 15

6.7 Calibration and cost-sensitive threshold . . . 18

6.8 Drift over the 12 future weeks . . . 19

6.9 The challenge set . . . 20

6.10 What protects, and the verdict . . . 20

7. Step 5: deployment 21

8. Reflections: what AutoML did well, and where it failed 24

8.1 What AutoML did well . . . 24

8.2 Where AutoML failed or misled . . . 25

8.3 What we would do differently . . . 25

9. Limitations 25

10. Reproducibility 27

10.1 Hardware . . . 27

10.2 Commands . . . 27

11. References 28

Appendix 29

A. Full AutoML leaderboards . . . 29

B. Tuned hyperparameters . . . 29

C. Experiment log . . . 30

D. Where to find everything . . . 30

Abstract

We train malware detectors on static features of Windows 64-bit PE files from EMBER2024 and compare manual hyperparameter tuning of LightGBM (grid, random and Bayesian search with Optuna) against three AutoML tools (FLAML, AutoGluon and H2O AutoML) under one fixed time budget of 1200 s each. Every model is scored on two held-out sets: *test_random* (the same 52 weeks as training) and *test_future* (12 strictly later weeks). The tuned LightGBM reaches ROC-AUC 0.9991 on test_random and 0.9981 on test_future (gap 0.0010 [0.0007, 0.0014]); the AutoML winners reach 0.9970 to 0.9980 on test_future: AutoGluon statistically tied the tuned LightGBM; FLAML and H2O AutoML were slightly but significantly worse (by at most 0.0011 AUC); none beat it (paired bootstrap). The random split overstates ranking quality only slightly; the larger overstatement comes from the 50/50 class mix: at a realistic 1% malware prevalence only 44.7% of the tuned model's alarms on test_future are real. On the EMBER2024 challenge set (malware that antivirus engines first missed) it detects only 53.7%. When a post-hoc label proxy (*detection_ratio*) is added as a feature, every tool uses it (4 of 4 models rank it first), reports a near-perfect validation score and gives no warning. AutoML came within 0.0011 test_future AUC of expert tuning in a fraction of the effort, but it did not protect against leakage or drift: those protections were human ones (column provenance, a temporal hold-out, a realistic base rate). The best model is served with FastAPI and Streamlit.

1. Problem statement and research question

Static malware detection decides from a file's bytes and PE structure, without running it, whether a Windows executable is malicious. It is a natural binary classification task, but its published scores are often measured on random splits, where training and test files come from the same weeks. In deployment a detector always faces files that are newer than anything it was trained on, and the malware share of real traffic is far below the 50% of a benchmark.

Research question. How much do random-split scores overstate real-world performance for malware detection, and do AutoML tools protect against, or fall for, leakage and temporal drift?

We answer it in four steps that follow the course rubric:

- **Step 2 (baseline and tuning):** LightGBM default and logistic regression baselines, then the same LightGBM tuned by grid search, random search and Optuna with an equal number of configurations.
- **Step 3 (AutoML):** FLAML, AutoGluon and H2O AutoML on the same training split with the same budget, their leaderboards, and a comparison with manual tuning on score, time and interpretability.
- **Step 4 (critical evaluation):** random vs temporal split, two deliberate leaky features, bias and prevalence, SHAP explanations, calibration, a cost-sensitive threshold, drift and a hard challenge set.
- **Step 5 (deployment):** export of the best model, a FastAPI service, a Streamlit demo and a model card.

Ground rules: one seed (42) everywhere; no model is trained, tuned, early-stopped or thresholded on test_random or test_future, and no configuration is selected on them (the one decision that reads test scores, which finished model to deploy, is stated openly in section 7); every table and figure is generated by a script from the experiment log or saved artefacts.

2. Dataset

We use **EMBER2024** (Joyce et al., KDD 2025), the successor of the original EMBER benchmark, restricted to its Win64 PE slice. Each weekly file holds static features extracted by the *thrember* extractor (version 0.1.0, commit `0ef753e81d98`). The data are published on Hugging Face (`joyce8/EMBER2024`) under the Apache-2.0 license.

Table 1. Dataset facts (from reports/data_check.json).

item	value
Source	Hugging Face joyce8/EMBER2024: Win64_train.zip, Win64_test.zip, challenge.zip
Feature extractor	thrember 0.1.0 (commit 0ef753e81d98)
License	Apache-2.0
Training period	52 weekly files, 2023-09-24 to 2024-09-21 (weeks 0-51)
Test period	12 weekly files, 2024-09-22 to 2024-12-14 (weeks 52-63)
Raw lines	1,040,000 (training period) + 240,000 (test period)
Unique files after dedup	520,000 + 119,993
Feature vector	2,568 float32 columns in 12 families
Labels	0 = benign, 1 = malicious (only 0/1 present; no unlabelled rows)

2.1 The duplicate finding

While building the feature cache we found that every weekly file lists each sample twice: 520,000 of 1,040,000 training-period lines and 120,000 of 240,000 test-period lines are in-file sha256 duplicates, plus 7 duplicates across test files. We deduplicate globally by sha256 (first occurrence kept) *before* any split. Without that, a random split would put copies of the same file into train and test and inflate test_random further. The checks in data_check.json confirm zero sha256 overlap between the splits.

2.2 Splits and class balance

The brief asks for a stratified subsample of 150k-250k rows, the same for every method. From the training period we draw **train** (160,000) and **test_random** (40,000), stratified by label x week; from the 12 later weeks we draw **test_future** (40,000). test_future starts strictly after the last training file. Step 2 tunes on a 60,000-row subset of train with 3 persisted CV folds.

Table 2. Splits and class balance (reports/tables/class_balance.csv, data/splits/splits_info.json). Classes are 50/50 by construction, because each week of EMBER2024 holds equal numbers.

split	rows	benign	malicious	malicious share	weeks	dates
train	160,000	80,003	79,997	50.00%	52	2023-09-24 to 2024-09-21
test_random	40,000	19,996	20,004	50.01%	52	2023-09-24 to 2024-09-21
test_future	40,000	19,997	20,003	50.01%	12	2024-09-22 to 2024-12-14

2.3 Feature families

Table 3. The 12 thrember feature families (offsets from reports/data_check.json).

family	columns	column range	content
general	7	0-6	file size, virtual size, imports/exports counts and similar
histogram	256	7-262	byte value histogram (256 bins)
byteentropy	256	263-518	joint byte/entropy histogram
strings	177	519-695	printable strings: counts, character frequencies, special patterns
header	74	696-769	COFF and optional header fields (incl. COFF timestamp)
section	224	770-993	section names, sizes, entropy, characteristics
imports	1282	994-2275	imported libraries and functions (hashed)
exports	129	2276-2404	exported functions (hashed)
datadirectories	34	2405-2438	PE data directory sizes and addresses
richheader	33	2439-2471	Rich header (compiler/linker) entries
authenticode	8	2472-2479	code-signing information (incl. signing times)
pefilewarnings	88	2480-2567	warnings raised by the pefile parser

Two metadata columns are **not** features: *detection_ratio* (the share of antivirus engines that flagged the file; its ROC-AUC against the label is 1.0000, because the label is derived from it) and *first_submission_date*. Step 4 adds each of them as a deliberate leak. Time proxies already inside X are the COFF timestamp (column 696) and the authenticode signing-time columns.

3. Methodology and workflow

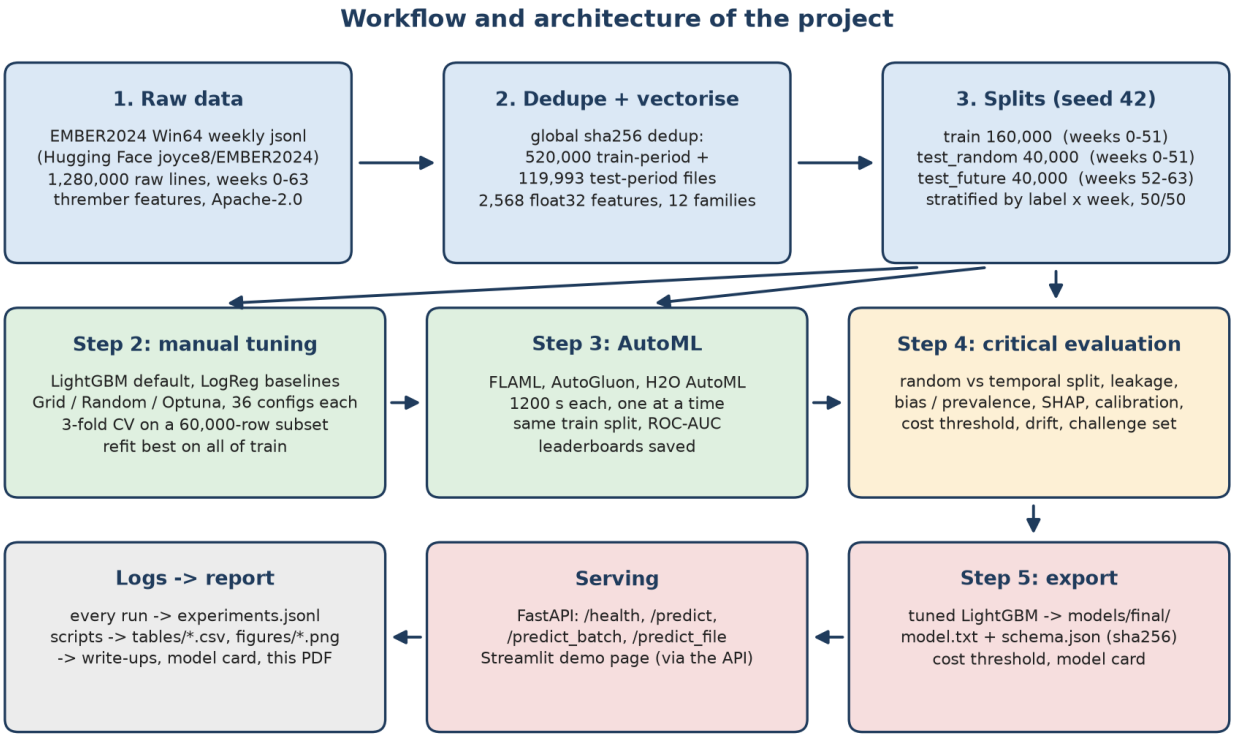


Figure 1. Workflow and architecture. Every run appends a record to reports/experiments.jsonl; scripts turn the log and the saved artefacts into the tables, figures and this report.

3.1 Design decisions

- **Model family for manual tuning: LightGBM.** Gradient-boosted trees are the standard strong baseline on EMBER, handle 2,568 unscaled numeric features, and train fast enough for 36 CV configurations per tuner. Logistic regression (on standardised features) is the linear reference.

- **Equal budget for the tuners:** grid, random and Optuna each evaluate 36 configurations, scored by the same 3-fold CV on the same 60,000-row subset of train, with early stopping inside the folds; the best configuration by mean CV AUC is refit on all of train.
- **Equal budget for the AutoML tools:** 1200 s of search each (the brief's 20 minutes), the same train split, ROC-AUC as the metric, run one after another on an otherwise idle machine. Wall-clock time is logged, including overruns.
- **Metrics:** ROC-AUC, F1 and accuracy at 0.5 (the rubric), plus TPR at 1% FPR (the operating point a malware analyst cares about). 95% confidence intervals come from a bootstrap with 1000 resamples (seeds 42 and 43, one per test set) or Wilson intervals for rates; model differences use paired bootstraps on the same rows.
- **Test discipline:** test_random and test_future are scored once per final model. Thresholds are chosen on train out-of-fold scores, never on a test set.
- **Reproducibility:** SEED = 42 everywhere; persisted splits and CV folds; every table and figure is rebuilt from the log by a `--tables-only` entry point.

4. Step 2: baseline and hyperparameter tuning

Two baselines (LightGBM with default parameters, logistic regression) and three tuners over one LightGBM search space. Each tuner ran 36 configurations; the time column is the whole search. Scores are for the refit of the best configuration on all 160,000 training rows.

Table 4. Step 2 results (reports/tables/tuning_summary.csv). CV AUC is the mean over the 3 folds of the 60k tuning subset; test columns are for the refit on all of train.

method	trials	CV AUC	AUC random	AUC future [95% CI]	F1 future	acc. future	TPR@1% FPR future	search + refit time
LightGBM default	-	0.9968	0.9978	0.9967 [0.9963, 0.9971]	0.9744	0.9744	95.1%	- + 25 s
LogReg	-	0.9842	0.9929	0.9297 [0.9265, 0.9329]	0.9172	0.9205	81.7%	- + 20 s
LightGBM grid	36	0.9977	0.9991	0.9980 [0.9977, 0.9984]	0.9827	0.9828	97.4%	113.3 min + 196 s
LightGBM random	36	0.9977	0.9990	0.9980 [0.9976, 0.9983]	0.9827	0.9828	97.5%	89.2 min + 96 s
LightGBM tuned (Optuna)	36	0.9978	0.9991	0.9981 [0.9978, 0.9984]	0.9838	0.9839	97.6%	74.3 min + 107 s

Findings. All three tuners beat the default LightGBM on test_future AUC (3 of 3 paired differences significant, by 0.0012 to 0.0014). Between the tuners the differences are tiny: 1 of 3 pairwise test_future AUC differences excludes zero (reports/tables/tuning_pairwise.csv), so with this budget the search method matters far less than tuning at all. LightGBM tuned (Optuna) had the best CV AUC (0.9978) and was therefore chosen as *the tuned model* for the rest of the project, a choice made on CV only. Search times: LightGBM grid 113.3 min, LightGBM random 89.2 min, LightGBM tuned (Optuna) 74.3 min (Figure 3 shows the best CV AUC against the number of configurations tried).

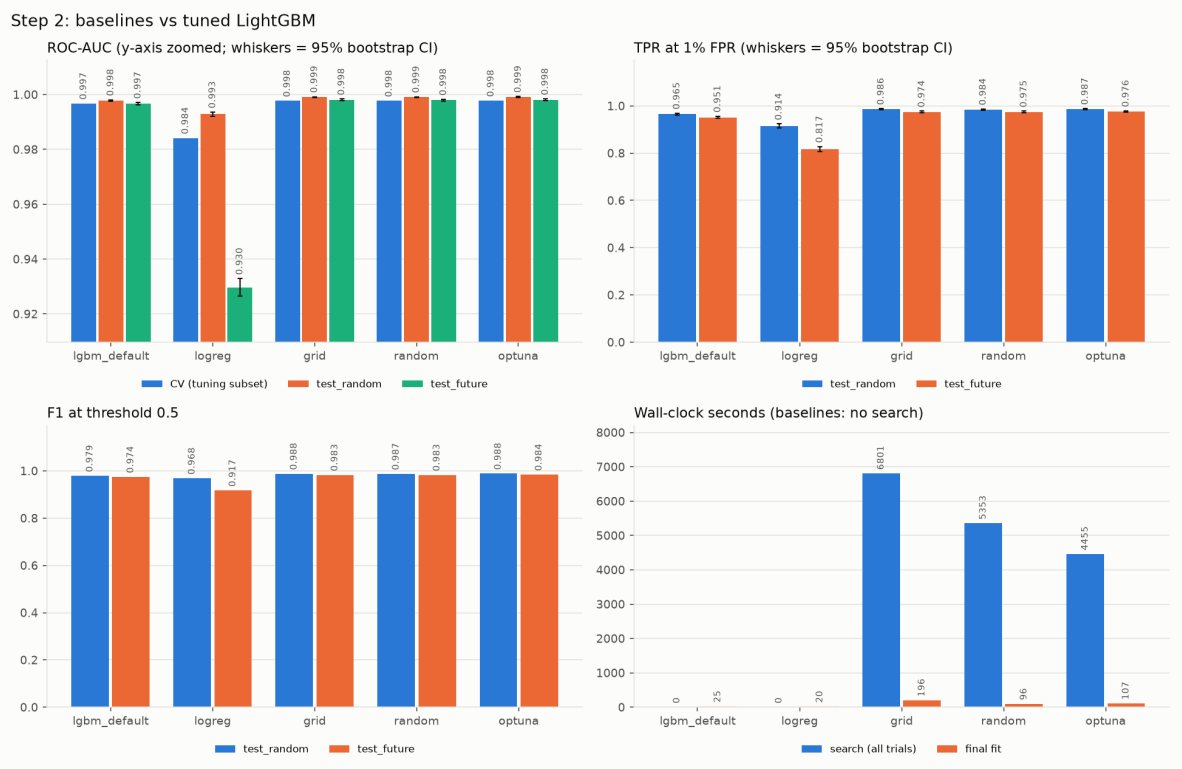


Figure 2. Step 2 comparison of the tuners and baselines: scores with 95% CIs on both test sets and the search time (reports/figures/tuning_compare.png).

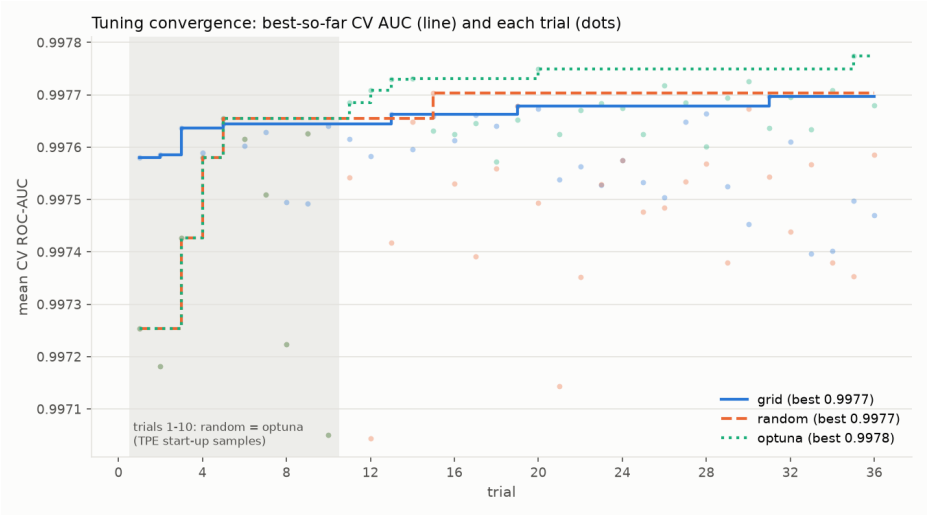


Figure 3. Best CV AUC so far against the number of configurations evaluated, per tuner (reports/figures/tuning_convergence.png).

5. Step 3: AutoML and tool comparison

FLAML, AutoGluon (medium_quality preset) and H2O AutoML were each given the same train split (160,000 rows x 2,568 features), ROC-AUC as the metric, 1200 s of search and 16 threads on the laptop, one after another in separate processes. Each tool chose its own internal validation inside train; neither test set was loaded before a tool's fit had finished.

Table 5. AutoML vs manual (reports/tables/automl_vs_manual.csv). For the tuned LightGBM the wall time covers the whole Step 2 search of its tuner plus the refit.

method	winning model	budget	wall time	AUC random	AUC future [95% CI]	F1 future	acc. future	TPR@1% FPR future
FLAML	xgb_limitdepth	1200 s	25.7 min	0.9986	0.9976 [0.9972, 0.9979]	0.9786	0.9787	96.5%
AutoGluon	WeightedEnsemble_L2	1200 s	20.2 min	0.9991	0.9980 [0.9977, 0.9984]	0.9834	0.9835	97.6%
H2O AutoML	StackedEnsemble_BestOfFamily_1	1200 s	24.8 min	0.9984	0.9970 [0.9966, 0.9974]	0.9771	0.9771	96.1%
LightGBM tuned (Optuna)	LightGBM, 609 trees	-	76.0 min	0.9991	0.9981 [0.9978, 0.9984]	0.9838	0.9839	97.6%
LightGBM default	lightgbm	-	0.4 min	0.9978	0.9967 [0.9963, 0.9971]	0.9744	0.9744	95.1%
LogReg	logistic_regression	-	0.3 min	0.9929	0.9297 [0.9265, 0.9329]	0.9172	0.9205	81.7%

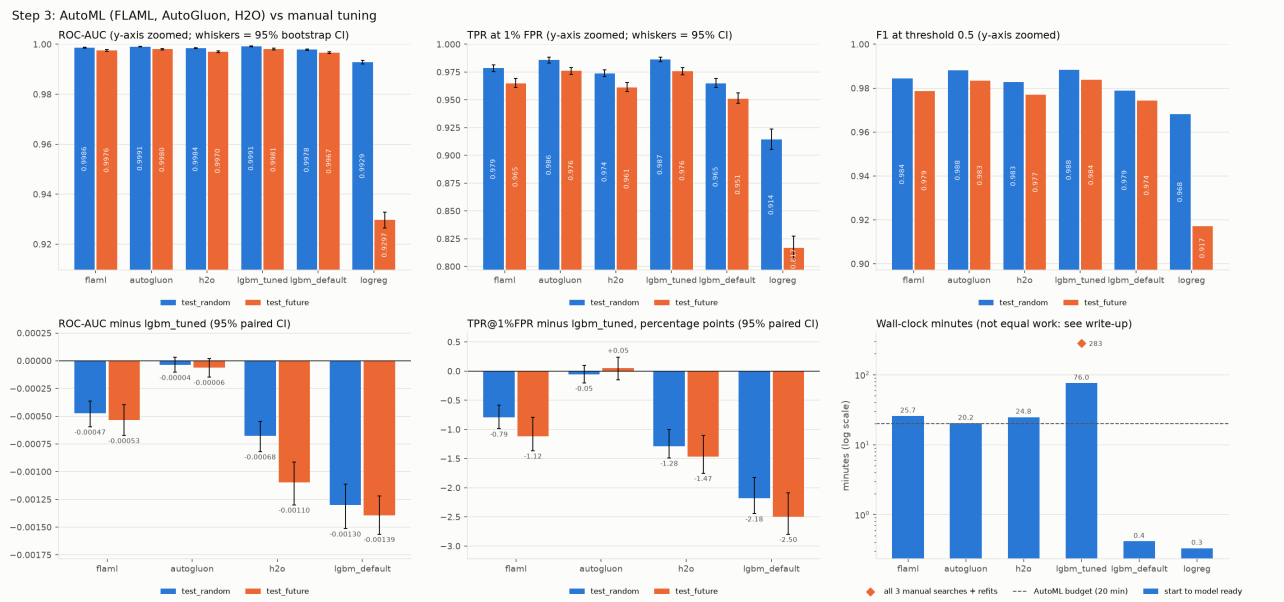


Figure 4. AutoML tools against the manual models on both test sets, with 95% bootstrap CIs (reports/figures/automl_vs_manual.png).

Paired test_future AUC difference against the tuned LightGBM (reports/tables/automl_pairwise.csv): FLAML -0.0005 [-0.0007, -0.0004] (significant); AutoGluon -0.0001 [-0.0001, 0.0000] (not significant); H2O AutoML -0.0011 [-0.0013, -0.0009] (significant); LightGBM default -0.0014 [-0.0016, -0.0012] (significant).

Time. Each tool's search stopped at the budget, but FLAML's final refit on all rows and H2O's stacked ensemble came on top: wall time minus budget was FLAML +342 s, AutoGluon +15 s, H2O AutoML +287 s. The manual route needed 4.7 h of search and refit for all three tuners together, against well under half an hour per AutoML tool.

5.1 Leaderboards

Summaries of the saved leaderboards (reports/leaderboards/); the full lists are in Appendix A.

Table 6. FLAML: top 4 estimators by validation AUC.

FLAML estimator	trials	best validation AUC	rows in best trial
xgb_limitdepth	17	0.9985	144,000
extra_tree	26	0.9968	144,000
lgbm	25	0.9955	10,000
catboost	2	0.9942	10,000

Table 7. AutoGluon: top 4 models by validation AUC.

AutoGluon model	validation AUC	fit time	stack level
LightGBMXT	0.9994	394 s	1
WeightedEnsemble_L2	0.9994	394 s	2
LightGBM	0.9992	298 s	1
RandomForestEntr	0.9981	162 s	1

Table 8. H2O AutoML: top 4 models (leaderboard on its validation frame).

H2O model	algorithm	leaderboard AUC	training time
StackedEnsemble_BestOfFamily_1	StackedEnsemble	0.9984	27 s
StackedEnsemble_BestOfFamily_2	StackedEnsemble	0.9984	18 s
StackedEnsemble_AllModels_1	StackedEnsemble	0.9984	35 s
GBM_1	GBM	0.9984	454 s

5.2 Preprocessing and interpretability

None of the tools engineered new features. FLAML's DataTransformer dropped the constant columns, fitted a median imputer (nothing to fill: no missing values) and scaled nothing; AutoGluon's feature generators dropped constant, duplicate and near-constant columns; H2O ignores constant columns inside each model. Read back from the saved models (reports/automl_comparison.md, reports/leaderboards/*_preprocessing.json):

Table 9. Preprocessing applied by each tool (table from reports/automl_comparison.md).

tool	features in	features used	dropped	constant in train	missing values in train	engineered features
FLAML	2568	2533	35	35	0	none
AutoGluon	2568	2515	53	35	0	none
H2O AutoML	2568	2533	35	35	0	none

Table 10. Interpretability of each final model (automl_vs_manual.csv, column interpretability).

method	what the final model is, and how it can be explained
FLAML	single xgb_limitdepth model (tree model: native importances and TreeSHAP)
AutoGluon	weighted ensemble with all weight on LightGBMXT, i.e. one model; it can be explained like any single model of that type
H2O AutoML	stacked ensemble of 2 base models, but its GLM metalearner gives weight only to GBM_1: in effect one model, explained by that model's variable importance
LightGBM tuned (Optuna)	single LightGBM model (609 trees): native importances and TreeSHAP
LightGBM default	single LightGBM model: native importances and TreeSHAP
LogReg	linear model on standardized features: one coefficient per feature

6. Step 4: critical evaluation

Hardware note. Steps 2-3 ran on the laptop (16-core Intel Core Ultra 9 285H, 63 GB RAM, Windows, 16 threads). Step 4 ran on a 4-core, 15.7 GiB RAM Linux cloud VM; its leakage runs used a 600 s AutoML budget, and every with/without comparison inside Step 4 used the same machine and settings. Step 4 timings and AutoML scores are therefore not comparable with Step 3 timings. The challenge set was scored on the laptop (scoring fits nothing, so hardware does not change those numbers).

6.1 The answer in short

- **Ranking metrics barely move for the tree models.** From test_random to test_future the tuned LightGBM goes from AUC 0.9991 to 0.9981 (gap 0.0010 [0.0007, 0.0014]) and from TPR@1%FPR 98.7% to 97.6% (gap +1.07 pp [+0.67 pp, +1.46 pp]). The AUC gaps of all 7 tree models lie between 0.0010 and 0.0014; only LogReg, which cannot cap out-of-range feature values, loses much more (0.0632 AUC). By AUC alone, the random split overstates the tuned model's real-world performance only slightly.
- **The overstatement is in the operating point and the base rate, which AUC hides.** At the threshold fixed on train (t = 0.5666) the false-positive rate rises from 0.95% on test_random to 1.23% on test_future (+29% relative). On the 50/50 test sets precision is 99.0% / 98.8%; at a realistic 1% malware prevalence it is 51.2% [47.6%, 54.7%] on test_random and 44.7% [41.6%, 47.8%] on test_future, i.e. 121 false alarms per 10,000 files scanned (random split: 94). Reading the 50/50 test_random as 1%-prevalence traffic changes precision by -47.87 pp; moving from test_random to test_future at that prevalence changes it by -6.50 pp: the 50/50 test mix is the larger source of optimism.
- **The hard tail is invisible to both test sets.** On the 814 Win64 challenge-set files (malware that antivirus engines missed when first seen) the tuned model detects 53.7% [50.3%, 57.1%] at the same threshold, against 97.9% of ordinary test_future malware.
- **No AutoML tool protects against the label leak.** With `detection_ratio` added, 4 of 4 models (tuned LightGBM and the three AutoML tools) ranked it first, by their own importance or by SHAP, and scored test_future AUC 0.9999 to 1.0000; 0 log lines named the column and 0 tools dropped it. The future split does not expose this leak either: a post-hoc label proxy is just as available (and perfect) in later weeks.
- **AutoML does not protect against drift.** None of the 3 AutoML tools has a smaller AUC gap than the tuned LightGBM; a larger one: H2O AutoML (+0.0004 [+0.0002, +0.0007]) (paired bootstrap CI of the gap difference), `reports/tables/gap_vs_tuned.csv`. Every tool validated on a random split of `train` (the same weeks as its training rows): its own validation AUC was 0.9952 to 0.9994 on the clean runs, against test_future 0.9922 to 0.9980, and 1.0000 with the leak: a near-perfect score that no tool questioned.

6.2 Random vs temporal split

Table 11. Random vs future, saved Step 2-3 models at threshold 0.5 (reports/tables/random_vs_future.csv). Gap = test_random minus test_future.

model	AUC random	AUC future	AUC gap [95% CI]	TPR@1% FPR random	TPR@1% FPR future	TPR gap	FPR random (t=0.5)	FPR future (t=0.5)
LightGBM default	0.9978	0.9967	0.0011 [0.0006, 0.0016]	96.5%	95.1%	+1.39 pp	1.74%	2.42%
LogReg	0.9929	0.9297	0.0632 [0.0600, 0.0664]	91.4%	81.7%	+9.74 pp	3.15%	3.92%
LightGBM grid	0.9991	0.9980	0.0010 [0.0007, 0.0014]	98.6%	97.4%	+1.18 pp	0.99%	1.41%
LightGBM random	0.9990	0.9980	0.0011 [0.0007, 0.0014]	98.4%	97.5%	+0.98 pp	1.07%	1.51%
LightGBM tuned (Optuna)	0.9991	0.9981	0.0010 [0.0007, 0.0014]	98.7%	97.6%	+1.07 pp	1.04%	1.32%
FLAML	0.9986	0.9976	0.0011 [0.0007, 0.0015]	97.9%	96.5%	+1.39 pp	1.27%	1.97%
AutoGluon	0.9991	0.9980	0.0010 [0.0007, 0.0014]	98.6%	97.6%	+0.96 pp	1.03%	1.45%
H2O AutoML	0.9984	0.9970	0.0014 [0.0010, 0.0019]	97.4%	96.1%	+1.25 pp	1.59%	2.53%

Table 12. Which internal estimate predicts test_future best? Tuned LightGBM (reports/tables/cv_comparison.csv).

estimate	AUC	TPR@1%FPR	|AUC error| vs future	how it was computed
stratified_3fold_cv	0.9988	98.3%	0.0007	persisted StratifiedKFold(3, shuffle, seed 42) over all of train; source: shared with the explain job (reports/predictions/oof_lgbm_tuned.npz)
forward_cv_4fold	0.9975	96.6%	0.0006	fit weeks 0-11, validate weeks 12-21; fit weeks 0-21, validate weeks 22-31; fit weeks 0-31, validate weeks 32-41; fit weeks 0-41, validate weeks 42-51
test_random	0.9991	98.7%	0.0010	refit on all of train (saved Step 2 model), same weeks as train
test_future	0.9981	97.6%	0.0000	refit on all of train (saved Step 2 model), weeks 52-63: the target
step2_tuning_cv (60k subset, early stopping)	0.9978	-	0.0003	best trial's CV AUC from the Step 2 search log; optimistic (early stopping saw the folds)

- **The inputs do drift.** A classifier separates train files from future files with AUC 0.9383 (control, same weeks: 0.4973); the three time-stamp columns carry 53.8% of its gain (`reports/tables/gap_adversarial.csv`, `reports/tables/gap_adversarial_top_features.csv`). Trees cannot extrapolate: a value beyond the last split lands in the same leaf as the newest training values, which limits what drift in such columns can do to the detector.
- **Removing the time proxies inside X does not shrink the gap.** Without the COFF timestamp and the two signing-time columns the tuned model's AUC gap goes from +0.00101 to +0.00107 (`reports/tables/leakage.csv`, `ablate_time_proxies`), so they are not a hidden time leak that flatters test_random.
- **No label shift.** The malicious share is 50.00% / 50.01% / 50.01% in train / test_random / test_future (`reports/tables/class_balance.csv`), by construction, so the score cannot move through the class mix.
- **Short horizon, slow decay.** Over the 12 future weeks the weekly AUC slope is -0.00011 per week (`reports/tables/drift.csv`); a model retrained on only the last 12 training weeks scores 0.9973 on test_future, against 0.9981 for the model trained on all 52: at this horizon, more data beats recency. At equal size, recent weeks may be worth slightly more than random weeks 0-51: 0.9973 vs 0.9969 for a size-matched random sample (higher in 10 of 12 future weeks; suggestive only, with no paired CI on this difference and one seed).
- **Weekly stratification** makes test_random an interpolation test: every test_random file has training files from its own week, which is the main reason it is the optimistic one.

6.3 Deliberate leakage

Two leaky columns were added, one at a time, to tuned LightGBM and to each AutoML tool, which were retrained with and without the column at the same budget (600 s per AutoML run on the VM). A third variant removes the time proxies already inside X from the tuned model.

Table 13. Leakage experiments (reports/tables/leakage.csv). * = paired bootstrap CI excludes 0. 'Relied on leak' = the model ranked the column first by its own importance or by SHAP (for FLAML only the SHAP rank counts, because its own importance is a split count).

model	variant	AUC random	AUC future	delta future vs clean	leak rank (own / SHAP)	relied on leak	wall time (VM)
LightGBM tuned (Optuna)	clean	0.9991	0.9981				249 s
LightGBM tuned (Optuna)	leak_detection_ratio	1.0000	1.0000	+0.0019 *	#1 / #1	yes	122 s
LightGBM tuned (Optuna)	leak_first_submission	0.9991	0.9982	+0.0001 *	#45 / #28	no	254 s
LightGBM tuned (Optuna)	ablate_time_proxies	0.9990	0.9979	-0.0002 *			237 s
FLAML	clean	0.9990	0.9980				1021 s
FLAML	leak_detection_ratio	1.0000	1.0000	+0.0020 *	#2 / #1	yes	772 s
FLAML	leak_first_submission	0.9988	0.9978	-0.0002 *	#1 / #23	no	885 s
AutoGluon	clean	0.9991	0.9980				608 s
AutoGluon	leak_detection_ratio	0.9999	0.9999	+0.0019 *	#1 / #1	yes	406 s
AutoGluon	leak_first_submission	0.9991	0.9982	+0.0001 *	#3 / #28	no	608 s
H2O AutoML	clean	0.9959	0.9922				1011 s
H2O AutoML	leak_detection_ratio	1.0000	1.0000	+0.0078 *	#1 / #1	yes	958 s
H2O AutoML	leak_first_submission	0.9962	0.9931	+0.0009 *	#182 / #201	no	963 s

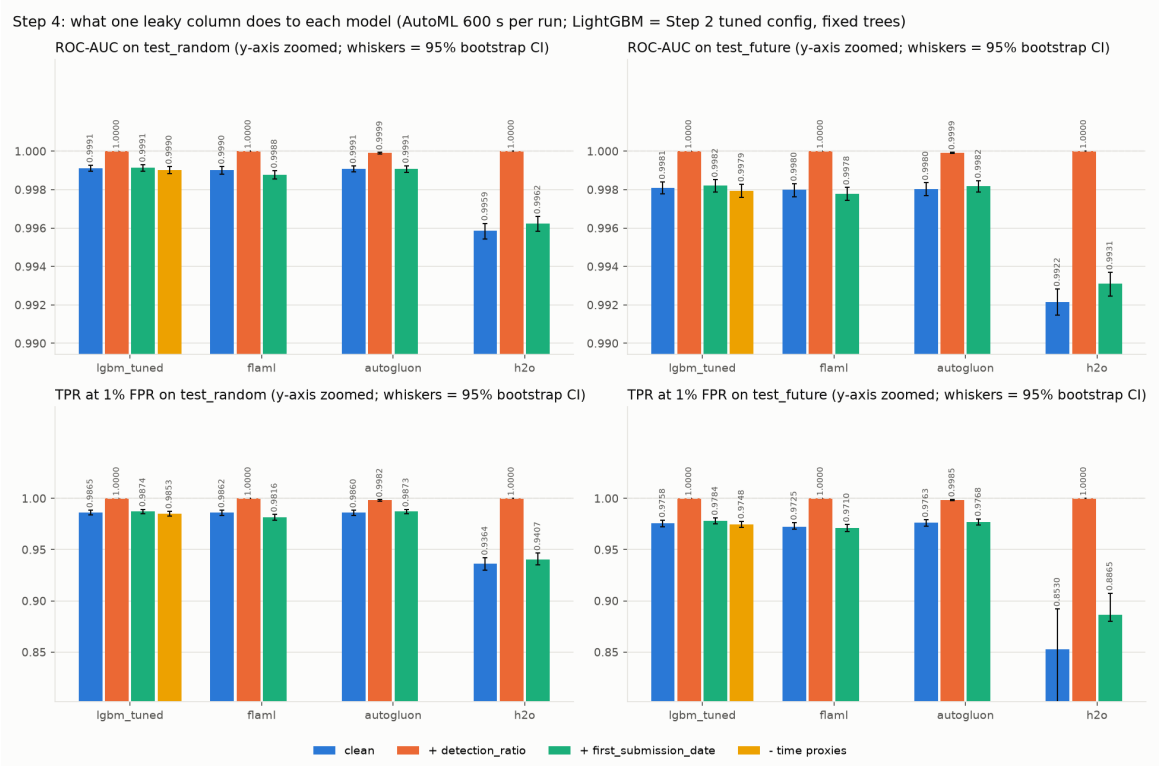


Figure 5. Score with and without each leak column, per model, both test sets (reports/figures/leakage_jump.png).

Each model was trained with and without one leak column at the same budget (600 s per AutoML run, on the VM). Delta = leak minus clean on the same test_future rows, * = paired-bootstrap CI excludes 0. Rank = the leak column's rank by the tool's own importance / by mean |SHAP|. Source: `reports/tables/leakage.csv`; the log scan is in `reports/leaderboards/leakage/log_scan.json`.

Relied on it = kept the column and ranked it first by its own importance or by SHAP. FLAML's own importance (`feature_importances_` of its LightGBM learner) is the split count, which favours continuous high-cardinality columns such as a date, so for FLAML only its SHAP rank counts.

- `detection_ratio` **is a label leak, and every tool took it.** It is the share of antivirus engines that flagged the file, i.e. the thing the label was made from. A column with AUC 1.0000 against the label (`reports/data_check.json`) is exactly what a search for the best score is built to find, so no tool can be expected to reject it from the data alone; only a human question about provenance ('is this known when the file first arrives?') catches it.
- `first_submission_date` **is mostly harmless here, but for a fragile reason.** It moved test_future AUC by at most 0.0009. For 4 of 4 models that move excludes 0, but only under test-set resampling: the CIs do not cover the run-to-run variation of a time-budgeted AutoML search. H2O's move (+0.0009) is smaller than the gap between H2O's two clean searches (test_future AUC 0.9970 in Step 3 vs 0.9922 in the Step 4 clean run), but those two runs differ in budget and machine, so this is only indirect evidence of search noise: the true run-to-run variation was not measured (one seed per run). So no effect of the date column on the AutoML tools is shown. Contrary to the plan's expectation (a time leak that helps under the random split and fails on test_future), the date did not inflate test_random AUC except for H2O AutoML, possibly within the same (unmeasured) search noise, so this run does not demonstrate a time leak under the random split. Every future date lies after the last training date, so for a tree the column is a constant on test_future; with a 50/50 class mix in every week it carries almost no label information. In data where the malware share changes over time, the same column would let a model learn the calendar.
- **The tools' own validation did not raise the alarm either:** with `detection_ratio` their internal validation AUC was 1.0000 (`reports/tables/automl_self_estimate.csv`). A near-perfect AUC on malware detection should itself be treated as a bug report.

6.4 Do AutoML tools protect against drift?

Table 14. Random-vs-future AUC gap compared with the tuned LightGBM's gap, paired bootstrap (reports/tables/gap_vs_tuned.csv).

model	kind	AUC gap	gap minus tuned gap [95% CI]	differs
LightGBM default	baseline	0.0011	+0.0001 [-0.0002, 0.0004]	no
LogReg	baseline	0.0632	+0.0622 [0.0590, 0.0653]	yes
LightGBM grid	tuned	0.0010	-0.0000 [-0.0001, 0.0001]	no
LightGBM random	tuned	0.0011	+0.0000 [-0.0001, 0.0002]	no
LightGBM tuned (Optuna)	tuned	0.0010	+0.0000 [0.0000, 0.0000]	no
FLAML	automl	0.0011	+0.0001 [-0.0001, 0.0002]	no
AutoGluon	automl	0.0010	+0.0000 [-0.0001, 0.0001]	no
H2O AutoML	automl	0.0014	+0.0004 [0.0002, 0.0007]	yes

Table 15. What each tool's own validation (a random split of train) said, against the test sets (reports/tables/automl_self_estimate.csv).

tool	run	variant	budget	machine	own valid. AUC	AUC random	AUC future	own minus future
FLAML	step3	clean	1200 s	16 threads, Steps 2-3 laptop	0.9985	0.9986	0.9976	+0.0009
AutoGluon	step3	clean	1200 s	16 threads, Steps 2-3 laptop	0.9994	0.9991	0.9980	+0.0014
H2O AutoML	step3	clean	1200 s	16 threads, Steps 2-3 laptop	0.9984	0.9984	0.9970	+0.0014
FLAML	step4	clean	600 s	4 threads, 4-CPU Linux VM	0.9952	0.9990	0.9980	-0.0027
AutoGluon	step4	clean	600 s	4 threads, 4-CPU Linux VM	0.9994	0.9991	0.9980	+0.0014
H2O AutoML	step4	clean	600 s	4 threads, 4-CPU Linux VM	0.9955	0.9959	0.9922	+0.0034
FLAML	step4	leak_detection_ratio	600 s	4 threads, 4-CPU Linux VM	1.0000	1.0000	1.0000	-0.0000
AutoGluon	step4	leak_detection_ratio	600 s	4 threads, 4-CPU Linux VM	1.0000	0.9999	0.9999	+0.0000
H2O AutoML	step4	leak_detection_ratio	600 s	4 threads, 4-CPU Linux VM	1.0000	1.0000	1.0000	+0.0000
FLAML	step4	leak_first_submission	600 s	4 threads, 4-CPU Linux VM	0.9953	0.9988	0.9978	-0.0025
AutoGluon	step4	leak_first_submission	600 s	4 threads, 4-CPU Linux VM	0.9994	0.9991	0.9982	+0.0012
H2O AutoML	step4	leak_first_submission	600 s	4 threads, 4-CPU Linux VM	0.9962	0.9962	0.9931	+0.0031

- AUC: none of the 3 AutoML tools has a smaller AUC gap than the tuned LightGBM; a larger one: H2O AutoML (+0.0004 [+0.0002, +0.0007]) (paired bootstrap CI of the gap difference). TPR@1%FPR: none of the 3 AutoML tools has a smaller TPR@1%FPR gap than the tuned LightGBM; none has a larger one (paired bootstrap CI of the gap difference).
- The AutoML winners are gradient-boosted trees or ensembles of them, so they share the tuned model's robustness to out-of-range values; a tool's search adds no protection of its own, because it only ever sees `train`.

6.5 Bias, imbalance and base rate

Table 16. Tuned LightGBM at its train 1%-FPR threshold, re-weighted to realistic malware prevalence (reports/tables/precision_both_sets.csv).

test set	prevalence	threshold	TPR	FPR	precision [95% CI]	false alarms / 10k	missed / 10k	precision at 50/50
test_random	1%	0.5666	98.6%	0.95%	51.2% [47.6%, 54.7%]	94	1	99.0%
test_random	5%	0.5666	98.6%	0.95%	84.5% [82.6%, 86.3%]	90	7	99.0%
test_random	10%	0.5666	98.6%	0.95%	92.0% [90.9%, 93.0%]	86	14	99.0%
test_future	1%	0.5666	97.9%	1.23%	44.7% [41.6%, 47.8%]	121	2	98.8%
test_future	5%	0.5666	97.9%	1.23%	80.8% [78.8%, 82.7%]	116	10	98.8%
test_future	10%	0.5666	97.9%	1.23%	89.9% [88.7%, 91.0%]	110	21	98.8%

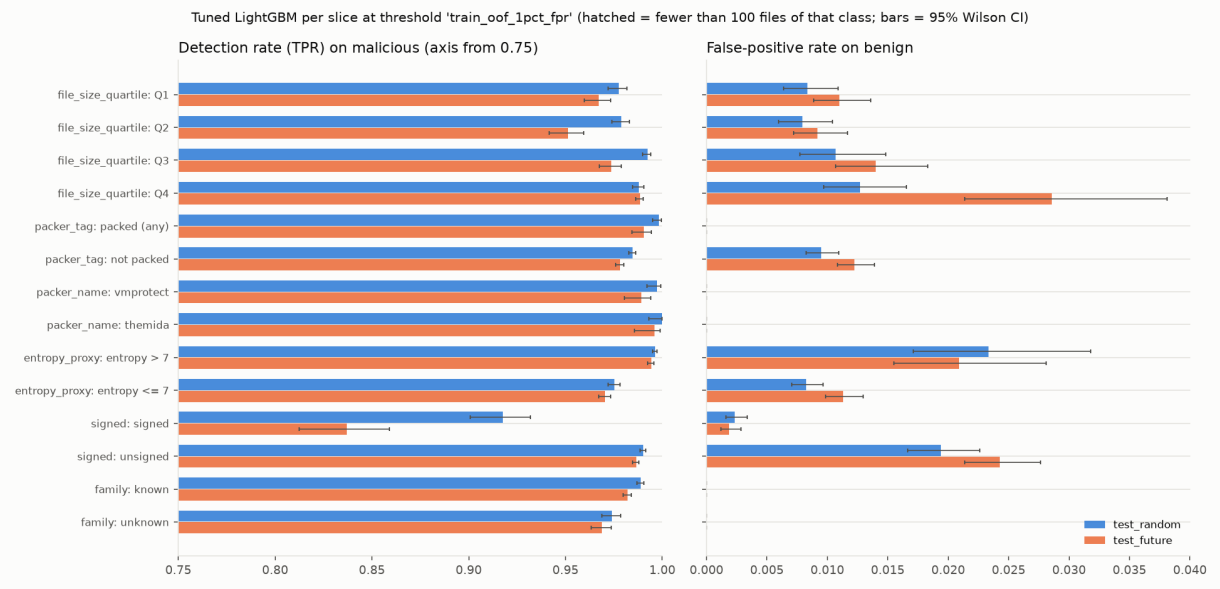


Figure 6. Per-slice detection rate and benign false-positive rate of the tuned model on test_future, with Wilson CIs (reports/figures/bias_slices.png).

- **Weakest slices on test_future** (slices with at least 100 files of the class): the highest benign FPR is `file_size_quartile` = Q4, 2.86% [2.14%, 3.81%] of 1,540 benign files; the lowest detection rate is `signed` = signed, 83.7% [81.3%, 85.9%] of 970 malicious files.

Table 17. Class weights on test_future (reports/tables/class_weight.csv). With 50/50 classes there is no imbalance to correct; reweighting only shifts the scores.

refit	AUC future	AUC minus no weights [95% CI]	TPR@1%FPR	F1 @0.5	FPR @0.5
tuned (saved Step 2 model)	0.9981	+0.0000 [0.0000, 0.0000]	97.6%	0.9838	1.32%
class_weight_none	0.9981	+0.0000 [0.0000, 0.0000]	97.6%	0.9838	1.32%
class_weight_balanced	0.9981	-0.0000 [-0.0001, 0.0000]	97.6%	0.9839	1.36%
class_weight_prior_10pct	0.9980	-0.0001 [-0.0002, -0.0000]	97.4%	0.9819	0.97%

6.6 Explainability (SHAP)

Table 18. TreeSHAP importance of the tuned LightGBM on test_future, grouped by EMBER feature family (reports/tables/shap_family_importance.csv). The summed share favours families with many columns; the grouped measure (mean |sum of the family's SHAP|) does not.

rank	family	columns	share of summed |SHAP|	grouped |SHAP|	top feature
1	strings	177	17.5%	1.69	strings: char_'_'_freq
2	imports	1282	15.1%	1.45	imports: function[hash 989]
3	histogram	256	12.8%	1.00	histogram: byte_0x00_freq
4	header	74	11.2%	1.65	header: coff.char.DLL
5	datadirectories	34	10.5%	2.00	datadirectories: EXPORT.size
6	section	224	9.8%	1.17	section: n_rx
7	byteentropy	256	8.2%	0.75	byteentropy: entropy_bin9/byte_hi_nibble0
8	authenticode	8	6.7%	1.77	authenticode: latest_signing_time
9	pefilewarnings	88	4.5%	1.18	pefilewarnings: warning[Suspicious flags set for section...]
10	richheader	33	2.0%	0.21	richheader: pair[hash 3]
11	exports	129	1.6%	0.32	exports: has_exports(0/128)
12	general	7	0.1%	0.02	general: size

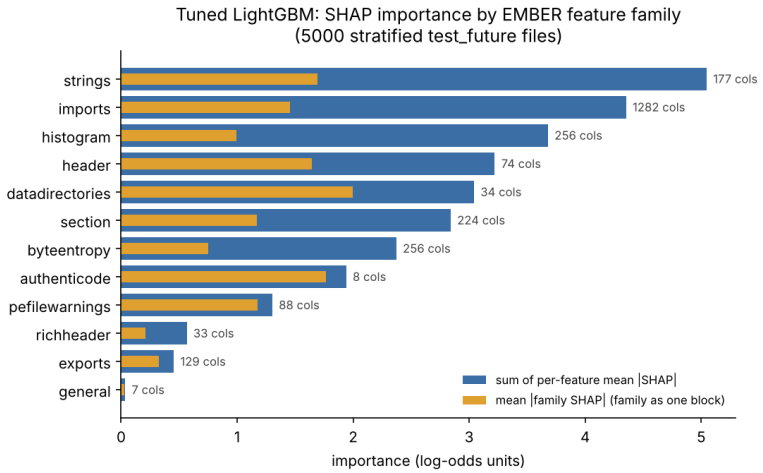


Figure 7. Mean |SHAP| by feature family (reports/figures/shap_family_bar.png).

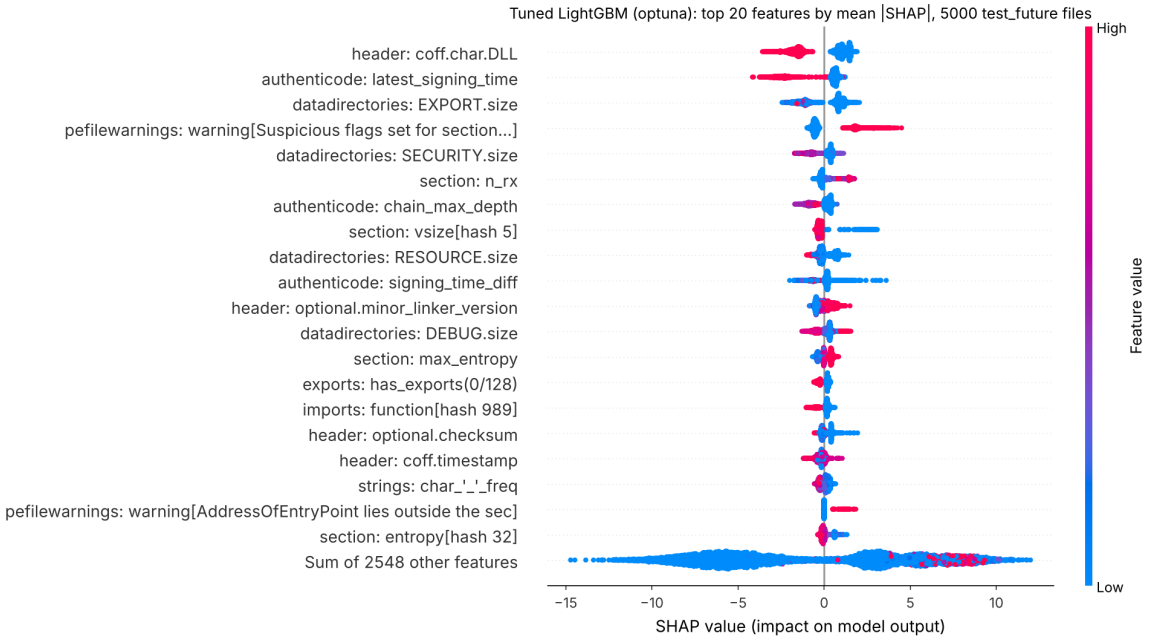


Figure 8. SHAP summary (beeswarm) of the top features of the tuned model (reports/figures/shap_beeswarm.png).

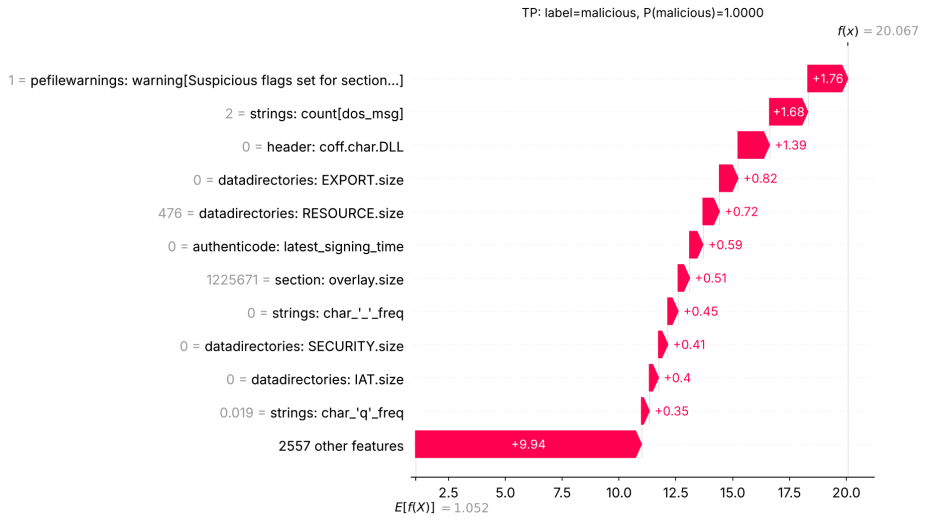


Figure 9. Local explanation of a true positive (week 52, P(malicious) = 1.0000): SHAP waterfall in log-odds (reports/figures/shap_waterfall_tp.png).

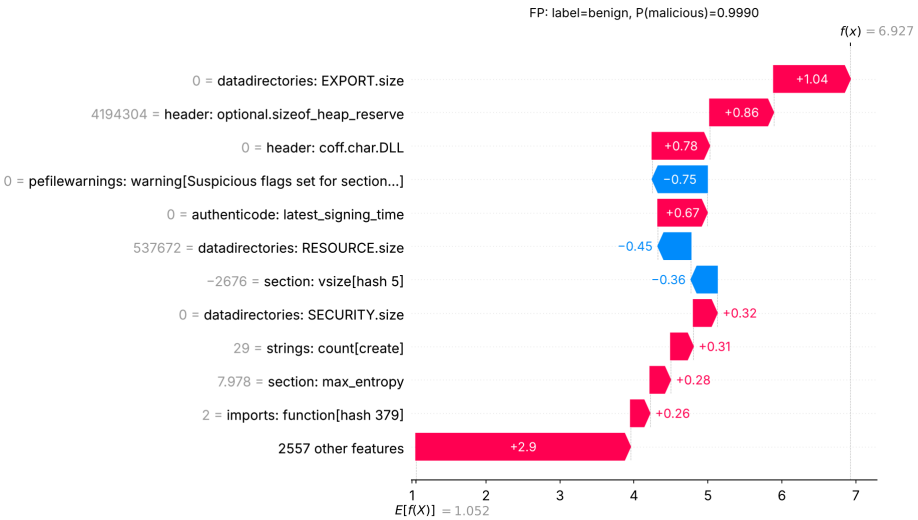


Figure 10. Local explanation of a false positive, a benign file flagged (week 62, P(malicious) = 0.9990): SHAP waterfall in log-odds (reports/figures/shap_waterfall_fp.png).

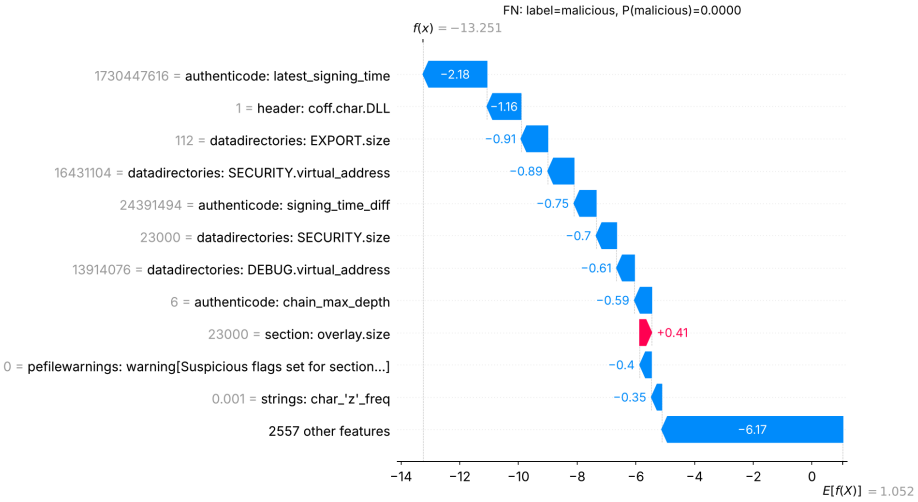


Figure 11. Local explanation of a false negative, malware the model missed (week 58, P(malicious) = 0.0000): SHAP waterfall in log-odds (reports/figures/shap_waterfall_fn.png).

Table 19. Cross-tool agreement of feature importance (reports/tables/cross_tool_agreement.csv). The measures differ by tool (SHAP, permutation importance, H2O varimp).

model A	model B	top-5 overlap	top-20 overlap	family rank correlation
LightGBM tuned (Optuna)	FLAML	4 of 5	14 of 20	0.99
LightGBM tuned (Optuna)	AutoGluon	3 of 5	8 of 20	0.81
LightGBM tuned (Optuna)	H2O AutoML	2 of 5	7 of 20	0.32
FLAML	AutoGluon	4 of 5	8 of 20	0.76
FLAML	H2O AutoML	3 of 5	10 of 20	0.33
AutoGluon	H2O AutoML	3 of 5	7 of 20	0.40

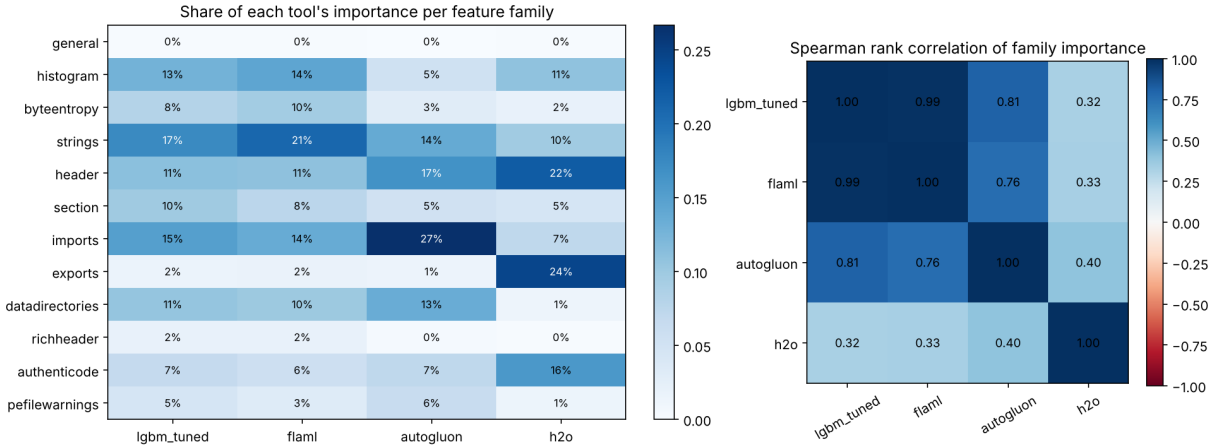


Figure 12. Feature-family importance shares across the tuned model and the AutoML winners (reports/figures/cross_tool_agreement.png).

6.7 Calibration and cost-sensitive threshold

Table 20. Calibration (reports/tables/calibration.csv). Isotonic and Platt calibrators were fitted on train out-of-fold scores.

model	scored on	Brier	log loss	ECE	AUC
LightGBM tuned (Optuna)	test_future	0.0132	0.0587	0.0085	0.9981
FLAML	test_future	0.0160	0.0576	0.0041	0.9976
AutoGluon	test_future	0.0129	0.0522	0.0056	0.9980
H2O AutoML	test_future	0.0173	0.0635	0.0062	0.9970
LogReg	test_future	0.0700	0.6227	0.0513	0.9297
lgbm_tuned+isotonic	test_future	0.0129	0.0484	0.0037	0.9981
lgbm_tuned+platt	test_future	0.0129	0.0485	0.0030	0.9981
lgbm_tuned (train OOF)	train_oof	0.0109	0.0502	0.0080	0.9988

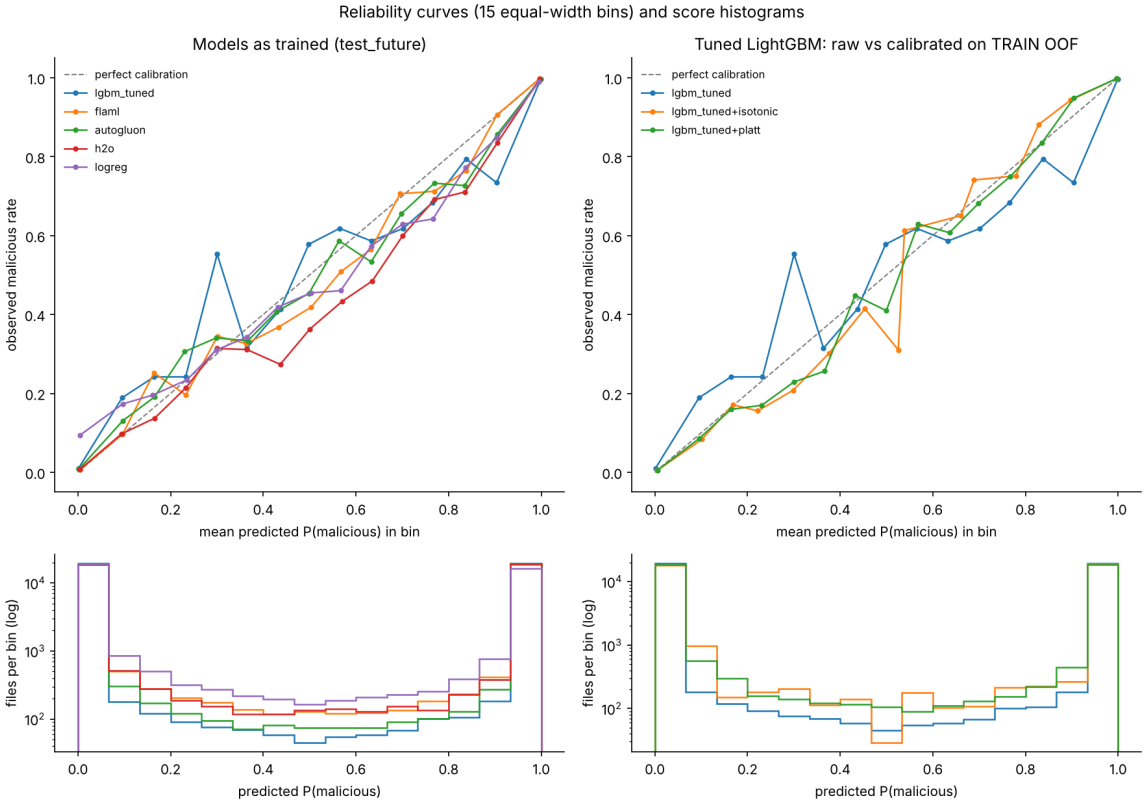


Figure 13. Reliability curves on test_future (reports/figures/calibration.png).

Table 21. Cost-minimising thresholds chosen on train out-of-fold scores, checked on test_future (reports/tables/threshold.csv).

cost FN:FP	threshold (train OOF)	recall OOF	FPR OOF	recall future	FPR future	cost / 1000 future	cost / 1000 at 0.5
1:1	0.4290	98.5%	1.18%	98.2%	1.48%	16.2	16.1
5:1	0.0384	99.3%	2.93%	99.2%	4.31%	42.7	54.2
10:1	0.0069	99.6%	4.93%	99.5%	7.43%	61.9	101.8
50:1	0.0008	99.8%	10.12%	99.7%	14.24%	139.9	482.9

The deployed threshold is the 10:1 one (t = 0.0069, models/threshold.json): a missed malware sample is assumed 10 times as costly as a false alarm. On test_future it gives recall 99.5% at FPR 7.43%. Caveat: it was chosen at 50/50 class balance and must be re-derived for a deployment with another prevalence.

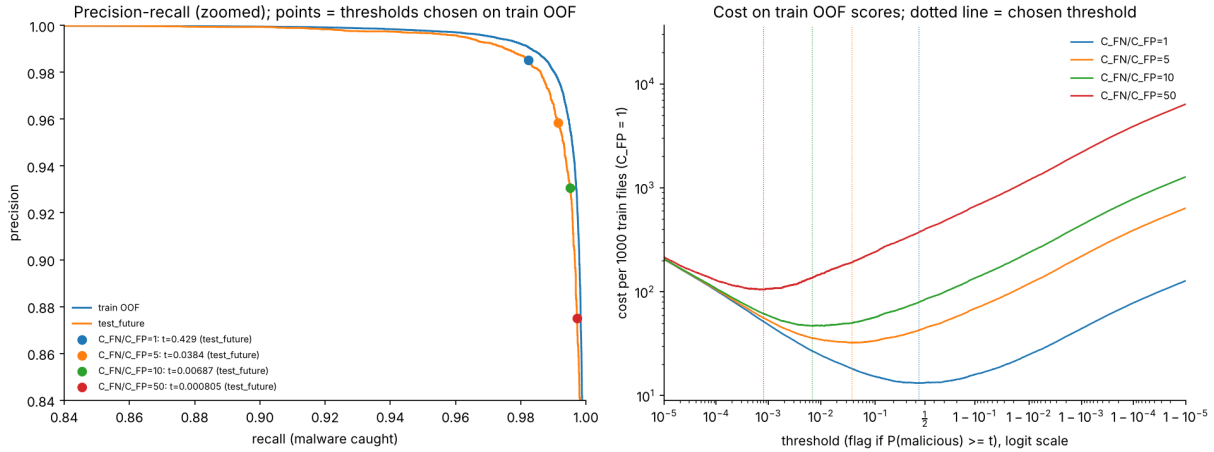


Figure 14. Precision-recall trade-off and expected cost against the threshold (reports/figures/pr_cost.png).

6.8 Drift over the 12 future weeks

The tuned configuration was retrained three ways and scored week by week on test_future: on all train weeks (expanding, 160,000 rows), on the last 12 training weeks only (sliding, 36,924 rows) and on a random sample of the same size from all weeks (size-matched). The weekly AUC slope of the expanding model is -0.00011 per week; its weekly AUC ranges from 0.9966 to 0.9991 (weeks 52-63). On the whole of test_future the AUC is 0.9981 (expanding), 0.9973 (sliding) and 0.9969 (size-matched): at this short horizon more data beats recency. At equal size recent weeks may help a little, but that is only suggestive: there is no paired CI on the sliding minus size-matched difference, and one seed.

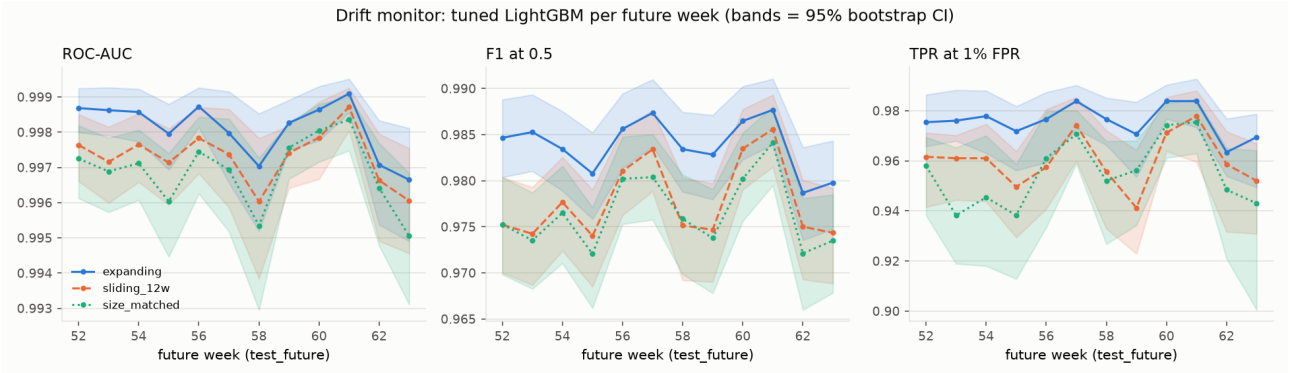


Figure 15. Weekly ROC-AUC, F1 and TPR@1%FPR on test_future for the expanding, sliding 12-week and size-matched models (reports/figures/drift.png).

6.9 The challenge set

EMBER2024's challenge set holds malware that antivirus engines missed when it was first seen. Its 814 Win64 files (all malicious, none in our splits) measure the hard tail that neither test set shows.

Table 22. Detection on the Win64 challenge set (reports/tables/challenge.csv).

model	threshold	t	files	detected	detection rate [95% CI]	TPR on test_future at same t
LightGBM default	0.5	0.5000	814	380	46.7% [43.3%, 50.1%]	97.3%
LightGBM default	train_oof_1pct_fpr	0.6668	814	271	33.3% [30.1%, 36.6%]	96.1%
LightGBM grid	0.5	0.5000	814	454	55.8% [52.3%, 59.2%]	98.0%
LightGBM random	0.5	0.5000	814	444	54.5% [51.1%, 57.9%]	98.1%
LightGBM tuned (Optuna)	0.5	0.5000	814	453	55.7% [52.2%, 59.0%]	98.1%
LightGBM tuned (Optuna)	train_oof_1pct_fpr	0.5666	814	437	53.7% [50.3%, 57.1%]	97.9%
LightGBM tuned (Optuna)	deployment	0.0069	814	650	79.9% [77.0%, 82.5%]	99.5%
LogReg	0.5	0.5000	814	423	52.0% [48.5%, 55.4%]	88.0%
LogReg	train_oof_1pct_fpr	0.9204	814	195	24.0% [21.2%, 27.0%]	81.4%
FLAML	0.5	0.5000	814	412	50.6% [47.2%, 54.0%]	97.7%
AutoGluon	0.5	0.5000	814	448	55.0% [51.6%, 58.4%]	98.1%
H2O AutoML	0.5	0.5000	814	436	53.6% [50.1%, 57.0%]	97.9%

6.10 What protects, and the verdict

- **A provenance review of every column, before training** (human): the only defence against `detection_ratio` in this project. It is cheap and no tool did it.
- **A temporal hold-out** (train on earlier weeks, test on later ones): it measures drift and makes a date column useless, but it cannot catch a post-hoc label proxy (section 6.3).
- **Thresholds and costs re-derived for the real base rate.** The 10:1 cost threshold in `models/threshold.json` (t = 0.0069), chosen on 50/50 train data, has FPR 7.43% on test_future and precision 11.9% at 1% prevalence (`reports/tables/prevalence.csv`): fine on a balanced test set, costly in real traffic.
- **Class weights did not help:** 0 class-weighted refit(s) improved test_future AUC beyond the CI (`reports/tables/class_weight.csv`). With 50/50 classes there is no imbalance to correct, and reweighting only shifts scores, which a threshold does as well.
- **Calibration fitted on train out-of-fold scores carries over to later weeks:** isotonic calibration changes the test_future Brier score by -0.0003 [-0.0006, -0.0001] (`reports/tables/calibration.csv`). It makes the probabilities more trustworthy, not the ranking.
- **Retraining on recent weeks only did not help at this horizon:** 0.9973 vs 0.9981 test_future AUC (`reports/tables/drift.csv`). The real remedy, retraining with labelled files from the deployment period, cannot be tested without using test_future for training.
- **A stress set of hard files** (the challenge set) is what showed the largest real-world weakness; neither random nor temporal splits of ordinary data show it.

For a tree model on EMBER2024 Win64, a random split overstates the ranking quality only a little (AUC 0.9991 vs 0.9981 on later weeks). The bigger overstatement comes from reading a 50/50 test set as if it were real traffic: at 1% prevalence only 44.7%, fewer than half, of the tuned model's alarms on test_future are real, and it misses 46% of the malware that antivirus engines missed at first. The AutoML tools' Step 3 models reach test_future AUC 0.9970 to 0.9980 (tuned LightGBM 0.9981), and they lose no less to drift: none of the 3 AutoML tools has a smaller AUC gap than the tuned LightGBM; a larger one: H2O AutoML (+0.0004 [+0.0002, +0.0007]) (paired bootstrap CI of the gap difference). They do not protect against leakage: given a post-hoc label column they all use it, report a near-perfect validation score, and say nothing. The protections that worked here were human ones: asking where each column comes from, testing on later data, and checking the operating point at a realistic base rate.

7. Step 5: deployment

Which model and why. The tuned LightGBM is deployed. Its configuration was fixed in Step 2 by CV AUC on train, before any test score existed. The final pick among the finished, frozen models is the one decision in this project that read test scores, so it is stated here: on test_future no AutoML tool was significantly better than the tuned model, and the tools statistically tied with it (paired AUC difference, tool minus tuned, CI includes 0; reports/tables/automl_pairwise.csv) were: AutoGluon -0.00006 [-0.00015, +0.00002]. The test scores therefore do not decide between the candidates. Practical reasons do: the tuned LightGBM is one plain model file with no framework runtime (no Java for H2O, no AutoGluon install), it can be explained exactly with TreeSHAP, and its threshold was fitted on its own train out-of-fold scores. Nothing was trained, tuned or thresholded on a test set.

Table 23. The exported model (models/final/schema.json).

item	value
model version	lgbm-optuna-c5aae42654cf
file / sha256	models/final/model.txt / c5aae42654cf73b4...
model	LightGBM gradient-boosted trees, binary objective, 609 trees, LightGBM 4.7.0
input	2,568 float32 thrember features (raw vector, no scaler needed)
threshold	0.006865 (cost ratio FN:FP = 10, from models/threshold.json)
export check	reproduces the saved Step 2 test_future scores on 40,000 rows, max |diff| = 0.0e+00
test_future	ROC-AUC 0.9981, TPR@1%FPR 97.6%, F1 0.9838

Serving. app/api.py is a FastAPI service. It verifies the model file against the sha256 in schema.json before serving, and offers:

- `GET /health`: status, model version, sha256 and threshold;
- `POST /predict`: one 2,568-value feature vector (validated with pydantic: length, finite values) returns P(malicious), the label at the threshold and, on request, the top TreeSHAP contributions;
- `POST /predict_batch`: several vectors at once;
- `POST /predict_file`: upload a PE file; it is vectorised in memory with thrember and never stored or executed.

app/streamlit_app.py is the demo page. It is one page that reads top to bottom for a non-specialist audience: how good the model is (recall and false-alarm rate on test_future at the deployed threshold), a choice of example (a caught malware file, a safe program, a missed malware file, a false alarm, or an uploaded PE file), the verdict scored by the API, the four feature families that pushed the decision most (SHAP sums, in plain words) and the limits. The raw score, the threshold and the top-10 SHAP columns are in a technical-detail section at the end. MODEL_CARD.md (generated from the same tables) documents data, metrics, intended and out-of-scope uses and known limits. DEMO.md is the 5-minute demo script. Deployment is local; cloud hosting is left to the student's own account.

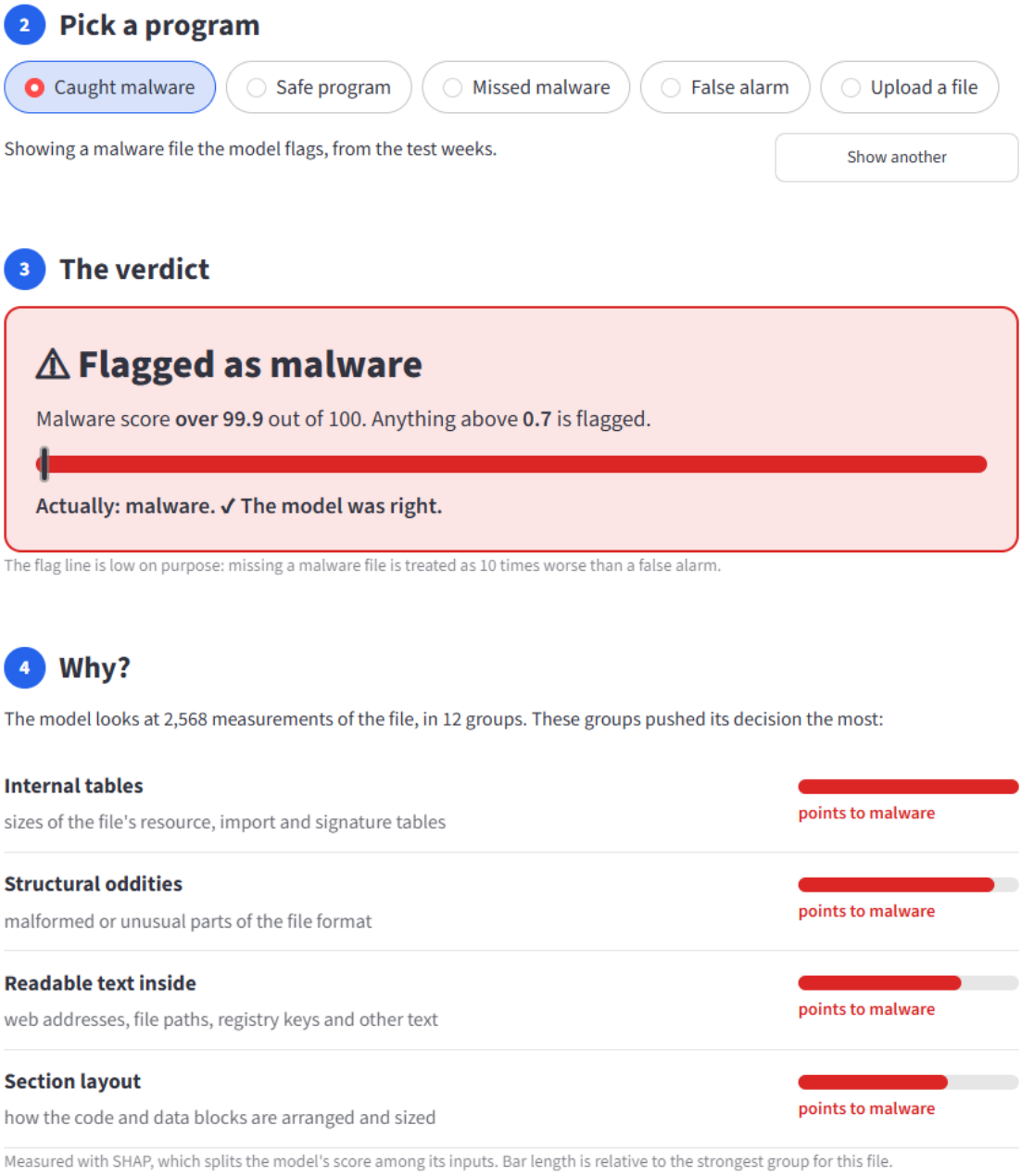


Figure 16. Streamlit demo: a malicious test_future file, correctly flagged, with the feature families that pushed the decision most. (reports/screenshots/a_malicious_correct.png, steps 2-4 of the page)

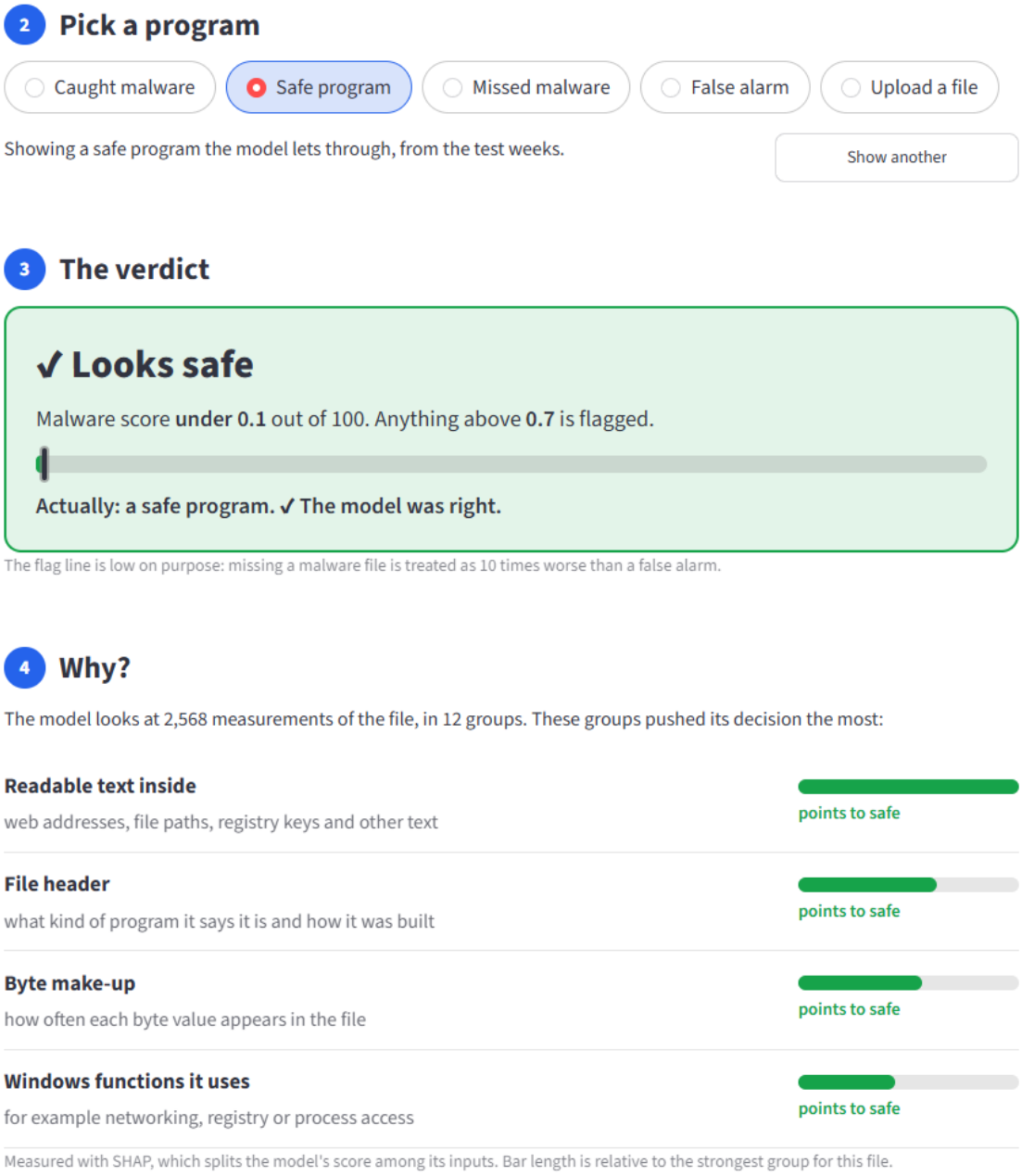


Figure 17. Streamlit demo: a benign test_future file, correctly passed. (reports/screenshots/b_benign_correct.png, steps 2-4 of the page)

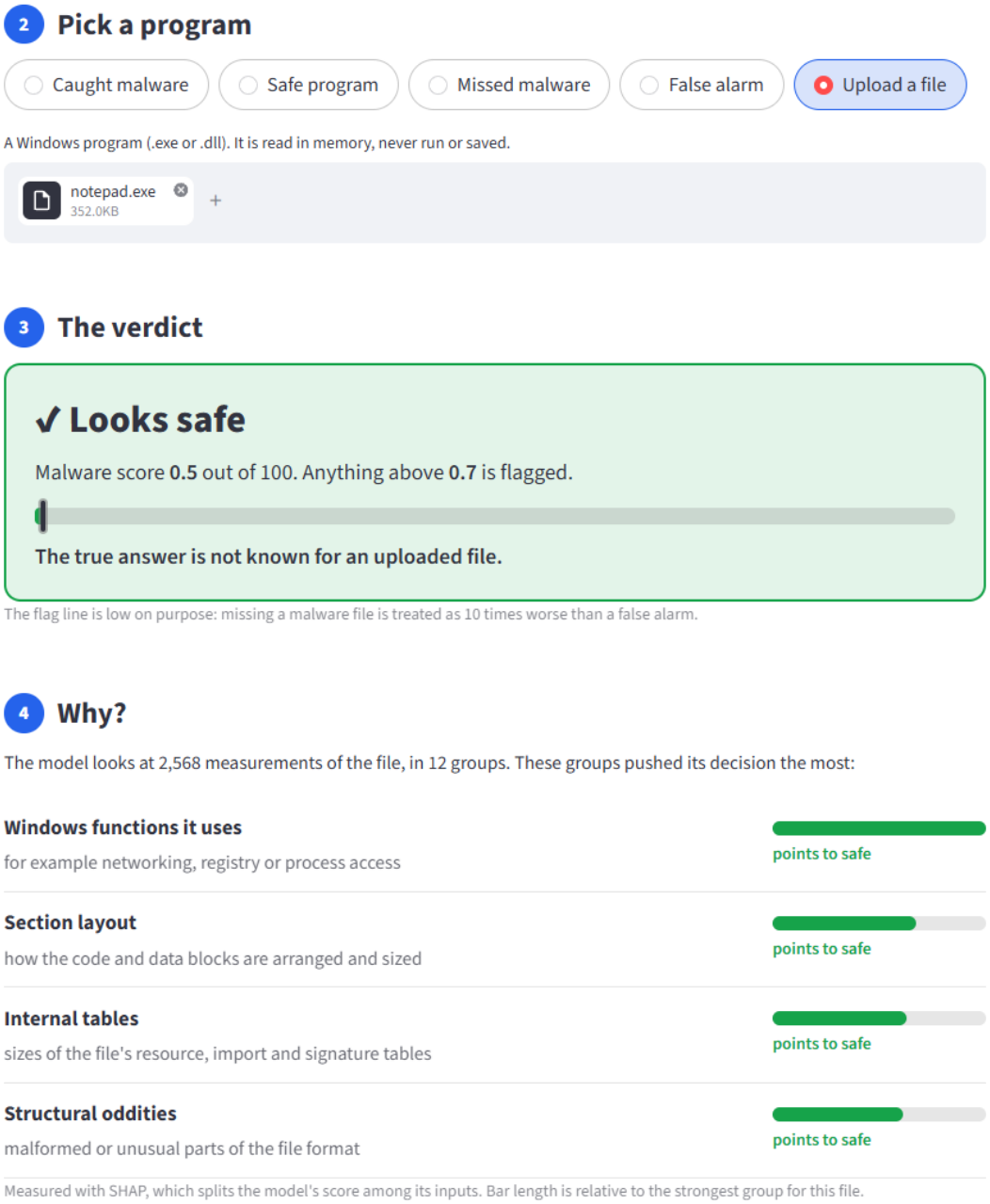


Figure 18. Streamlit demo: uploading a benign Windows binary (notepad.exe); the file is parsed in memory by the API and never executed. (reports/screenshots/c_upload_notepad.png, steps 2-4 of the page)

8. Reflections: what AutoML did well, and where it failed

8.1 What AutoML did well

- **Expert-level scores for little effort.** With one call and 1200 s each, the three tools reached test_future AUC 0.9970 to 0.9980, against 0.9981 for the tuned LightGBM, whose three searches and refits took 4.7 h together.
- **Sensible model choices.** All three converged on gradient-boosted trees (LightGBM, XGBoost or H2O GBM), the family a human would pick for this data, and FLAML grew its sample sizes so that cheap trials came first.
- **Transparent artefacts.** Every tool left a leaderboard, a log and importances that could be audited, which made the leakage and agreement analyses possible.

- **Mostly the same robustness to drift as the manual model.** FLAML and AutoGluon had random-vs-future gaps that match the tuned model's; H2O AutoML's is slightly larger (+0.0004 [0.0002, 0.0007], CI excludes 0); the winners are the same kind of model as the tuned one (reports/tables/gap_vs_tuned.csv).

8.2 Where AutoML failed or misled

- **It fell for the label leak.** With detection_ratio added, 4 of 4 models ranked it first and scored a near-perfect AUC; no tool warned or dropped the column. An optimiser for validation score cannot know that a column is only available after the fact.
- **Its own validation was a random split.** By default every tool validated on random rows of train (the same weeks), so its own estimate was above test_future AUC in 5 of 6 clean runs, and it reported 1.0000 with the leak.
- **Budgets are soft.** FLAML's refit and H2O's ensemble ran minutes past the budget, so 'equal budget' is equal search time, not equal compute.
- **Hardware sensitivity and repeatability.** On the 4-core VM the same tools searched fewer models in 600 s; H2O's test_future AUC dropped noticeably. A time-budgeted search is not exactly repeatable, even with a seed.
- **Interpretability depends on the winner.** FLAML and AutoGluon ended with one tree model, but H2O returned a stacked ensemble whose metalearner used one base model; explaining it took extra work.

8.3 What we would do differently

- Give each AutoML tool a time-ordered validation set (FLAML split_type='time', an H2O fold column, AutoGluon tuning_data) and compare it with the defaults.
- Review the provenance of every column before training; this is the only thing that stopped the label leak.
- Evaluate at a realistic prevalence and on a hard stress set from the start, not only on 50/50 test splits.
- Repeat the AutoML runs with several seeds on one machine to measure their run-to-run variation.

9. Limitations

This section is taken from reports/limitations.md, which scripts/run_critical_eval.py generates from the data checks and the Step 4 tables.

Data and features

- **Static features only.** Every model sees the 2,568 thrember features in 12 groups (general, histogram, byteentropy, strings, header, section, imports, exports, datadirectories, richheader, authenticode, pefilewarnings), computed from the file's bytes and PE structure without running it. Behaviour that only shows at run time (unpacking, network traffic, injected code) is invisible, and so is anything outside a Win64 PE file.
- **Packed and obfuscated files.** Packer tags in EMBER2024 exist only on malicious files, so the false-positive rate on packed *benign* software cannot be measured here; the model may simply have learned 'packed means malicious'. The feature-based proxy (whole-file entropy > 7) shows the cost: benign FPR on test_future is 2.09% for high-entropy files against 1.13% for the rest (`bias_slices.csv`, 1%-FPR train threshold).
- **Dataset age and recency.** Training files are from 2023-09-24 to 2024-09-21, test_future files from 2024-09-22 to 2024-12-14. The model has never seen anything newer, and the drift measured over 12 weeks says little about drift over the years between the data and any use of this model today.
- **Drift horizon.** All temporal results cover 12 future weeks. The weekly AUC slope (-0.00011 per week, `drift.csv`) comes with week-level CIs that overlap between the first (week 52) and the last (week 63) future week; a longer horizon could show more decay.

- **50/50 balance vs real prevalence.** The splits were built with 50.00% / 50.01% / 50.01% malware in train / test_random / test_future. Real traffic is mostly benign; at 1% prevalence the tuned model's precision on test_future would be 44.7% (`precision_both_sets.csv`). The real prevalence of the place of deployment is unknown, so 1%, 5% and 10% are scenarios, not measurements.
- **Subsampling.** After deduplication the official data has 520,000 training-period and 119,993 test-period files. We used 160,000 for train and 40,000 for test_random (from the training weeks) and 40,000 for test_future, stratified by label x week; Step 2 tuned on a 60,000-row subset with 3 folds. Larger samples would narrow the CIs, and could change close rankings between tuners and tools.
- **Label noise from AV-derived labels.** A label is a verdict of antivirus engines: in our cache no benign file has more than 2 detections and no malicious file fewer than 5 (149 benign training-period files have at least one), so files in the ambiguous middle are not in the data at all, which makes the task easier than real triage. Labels can also be wrong in ways the engines share, and they change as engines catch up: the challenge set consists of files the engines first missed.
- **Near-duplicates.** Deduplication removes exact copies only (same sha256); variants of one malware family remain on both sides of the random split, which is part of why test_random is easy (per-family slices in `bias_slices.csv`).
- **The duplicate rows in the raw data.** Each weekly file lists every sample twice (520,000 and 120,000 in-file duplicates in the training and test periods, plus 7 across test files). We deduplicate globally by sha256 before any split; without that, a random split would put copies of the same file into train and test and inflate test_random further. Anyone comparing with other EMBER2024 results should check whether they did the same.

Threats from the evaluation itself

- **Adversarial evasion was not tested.** Static features can be changed without changing behaviour (appended bytes, added imports or sections, a valid signature). The models lean on signature columns (`explainability.md`); detection of signed malware on test_future is 83.71% at the train 1%-FPR threshold, against 98.64% for unsigned malware, and the challenge set is detected at 53.7%. An attacker who knows this has an easy target.
- **One seed (42), one split per scheme.** The CIs cover test-set sampling only, not training randomness or another draw of the splits; time-budgeted AutoML searches are not repeatable, so deltas of about 1e-4 AUC between AutoML runs may be within run-to-run variation, which was not measured.
- **Many slices, no multiple-comparison correction.** Slice CIs are per slice; with dozens of slices some differences will look significant by chance. A test_random week has only about 385 benign files, so a single week's FPR there is very noisy.
- **Thresholds and calibrators come from smaller models.** They are fitted on train out-of-fold scores of fold models trained on two thirds of train, while the deployed model is trained on all of it (stated, not corrected; `threshold.csv` shows the effect at high cost ratios, where the test_future FPR is above the out-of-fold FPR).
- **The forward CV confounds recency with training-set size:** later folds are both closer in time and trained on more weeks. A size-matched control exists only for drift (`drift.csv`).
- **The AutoGluon leakage predictors were deleted** after scoring (about 2 GB each, little disk on the VM); their leaderboards, importances and test scores are saved, but those rows can be re-scored only by re-running them.
- **The challenge set is small and malicious only:** 814 Win64 files, so it measures misses, not false alarms.

Compute, hardware and budgets

- **CPU only, and two machines.** Steps 2-3 ran on a 16-core Intel Core Ultra 9 285H laptop (63 GB RAM, Windows) with 16 threads; Step 4 on a cloud VM (Intel(R) Xeon(R) Processor @ 2.10GHz, 4 logical CPUs, 15.7 GiB RAM, Linux 6.18.44-fc-v185). Every comparison inside a step ran on one machine with the same settings, but times are not comparable across steps, and an

AutoML tool searches fewer configurations per second on the VM. The challenge-set scoring alone was re-run on a 16-CPU Windows machine (the laptop), which holds every saved Step 2-3 model including AutoGluon's; scoring fits nothing, so the hardware does not change its result.

- **Memory forced runner changes on the VM.** The runner settings were the same for every leakage variant: a lean FLAML preprocessing and a smaller H2O heap without XGBoost. AutoGluon itself skipped memory-heavy models (only in the detection_ratio run; in the clean and date runs its first LightGBM used the whole budget), so on the VM it was in effect a single LightGBM model (per-run counts in `leakage.md`).
- **The 600 s leakage budget** is a stated deviation from the 1200 s of Step 3, to keep nine AutoML runs within a few hours. The clean reruns used the same budget and hardware, so each with/without-leak pair is equal-budget (not equal-compute: see below), but the clean scores are not the Step 3 scores (H2O: test_future AUC 0.9922 at 600 s on the VM vs 0.9970 at 1200 s on the laptop). Budgets are not hard limits: the AutoML runs ended -194 s to +421 s from their budget (wall minus budget, `leakage.csv`): the budget caps the search, and FLAML's final refit and H2O's data import, ensemble and scoring come on top, so the effective compute of one tool's variants differed by minutes (`leakage.md`, *Run times*).
- **Default AutoML settings.** We tested what a user gets by default (random-split validation, default presets); other settings, such as time-ordered validation, could behave differently.

10. Reproducibility

All code is in https://github.com/Turbo-Soggy/automl. One seed (42) is used for numpy, random, every model library and every split. Splits and CV folds are persisted under data/splits/. Every run appends a record (method, parameters, scores, time; Step 4 records also log the machine) to reports/experiments.jsonl, and every table, figure and write-up is regenerated from that log and the saved models.

10.1 Hardware

- **Steps 2-3 and Step 5** (tuning, AutoML, export and app): the student's laptop, 16-core Intel Core Ultra 9 285H, 63 GB RAM, Windows 11, CPU only, 16 threads.
- **Step 4** (critical evaluation): a 4-core, 15.7 GiB RAM Linux cloud VM. The leakage runs used a 600 s AutoML budget and memory-saving runner settings, identical for every with/without pair. The challenge-set scoring was re-run on the laptop (no fitting).
- Timings are only compared within one step (one machine).

10.2 Commands

```
python run_all.py --quick                  # smoke run on tiny samples (*_quick files)
python run_all.py                          # full run: data, tune, automl, eval
python scripts/run_tuning.py --tables-only # rebuild Step 2 tables, figures, write-up
python scripts/run_automl.py --tables-only # rebuild Step 3 tables, figure, write-up
python scripts/run_leakage.py --tables-only
python scripts/run_critical_eval.py        # Step 4 synthesis and limitations
python scripts/export_model.py             # Step 5 export + model card
python scripts/build_report.py             # this PDF (a few seconds)
python -m pytest -q                        # tests on splits, leakage, API
```

Here python is the project's virtual environment interpreter (.venv/Scripts/python.exe on Windows, .venv/bin/python on Linux). Python 3.12; all packages are pinned in requirements.txt (LightGBM, Optuna, FLAML, AutoGluon, H2O, SHAP, FastAPI, Streamlit, reportlab). Quick runs set EMBER_QUICK=1 and never overwrite full results.

Table 24. Tool versions logged with the runs.

tool	version
FLAML	2.7.0
AutoGluon	1.6.3
H2O AutoML	3.46.0.12
LightGBM (manual, deployed)	4.7.0

11. References

[1] R. J. Joyce, G. Miller, P. Roth, R. Zak, E. Zaresky-Williams, H. Anderson, E. Raff, J. Holt. *EMBER2024 - A Benchmark Dataset for Holistic Evaluation of Malware Classifiers.* KDD 2025. arXiv:2506.05074.

[2] H. S. Anderson, P. Roth. *EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models.* arXiv:1804.04637, 2018.

[3] G. Ke et al. *LightGBM: A Highly Efficient Gradient Boosting Decision Tree.* NeurIPS 2017.

[4] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama. *Optuna: A Next-generation Hyperparameter Optimization Framework.* KDD 2019.

[5] C. Wang, Q. Wu, M. Weimer, E. Zhu. *FLAML: A Fast and Lightweight AutoML Library.* MLSys 2021.

[6] N. Erickson et al. *AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data.* arXiv:2003.06505, 2020.

[7] E. LeDell, S. Poirier. *H2O AutoML: Scalable Automatic Machine Learning.* 7th ICML Workshop on AutoML, 2020.

[8] S. M. Lundberg, S.-I. Lee. *A Unified Approach to Interpreting Model Predictions.* NeurIPS 2017.

[9] S. M. Lundberg et al. *From local explanations to global understanding with explainable AI for trees.* Nature Machine Intelligence 2, 2020.

[10] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python.* JMLR 12, 2011.

[11] M. Mitchell et al. *Model Cards for Model Reporting.* FAT* 2019.

[12] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, L. Cavallaro. *TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time.* USENIX Security 2019.

[13] FastAPI (https://fastapi.tiangolo.com) and Streamlit (https://streamlit.io), software used for serving.

Appendix

A. Full AutoML leaderboards

Table 25. FLAML leaderboard (reports/leaderboards/flaml_leaderboard.csv).

rank	estimator	trials	best val. AUC	rows (best)	found at	time in trials
1	xgb_limitdepth	17	0.9985	144,000	996 s	242 s
2	extra_tree	26	0.9968	144,000	1056 s	155 s
3	lgbm	25	0.9955	10,000	246 s	242 s
4	catboost	2	0.9942	10,000	670 s	315 s
5	xgboost	11	0.9933	10,000	807 s	58 s
6	rf	20	0.9883	10,000	791 s	17 s
7	sgd	3	0.8099	10,000	147 s	36 s
8	lrl1	1	0.5018	10,000	1462 s	298 s

Table 26. AutoGluon leaderboard (reports/leaderboards/autogluon_leaderboard.csv).

model	val. AUC	fit time	predict time (val.)	stack level	features
LightGBMXT	0.9994	394 s	0.22 s	1	2515
WeightedEnsemble_L2	0.9994	394 s	0.22 s	2	1
LightGBM	0.9992	298 s	0.15 s	1	2515
RandomForestEntr	0.9981	162 s	0.15 s	1	2515
RandomForestGini	0.9979	256 s	0.15 s	1	2515
CatBoost	0.9930	53 s	0.04 s	1	2515

Table 27. H2O AutoML leaderboard (reports/leaderboards/h2o_leaderboard.csv).

model	algorithm	AUC	log loss	AUC-PR	training time
StackedEnsemble_BestOfFamily_1	StackedEnsemble	0.9984	0.0469	0.9986	27 s
StackedEnsemble_BestOfFamily_2	StackedEnsemble	0.9984	0.0469	0.9986	18 s
StackedEnsemble_AllModels_1	StackedEnsemble	0.9984	0.0469	0.9986	35 s
GBM_1	GBM	0.9984	0.0478	0.9986	454 s
DRF_1	DRF	0.9942	0.1102	0.9948	39 s
GBM_4	GBM	0.9929	0.2676	0.9934	33 s
GBM_3	GBM	0.9919	0.2073	0.9921	35 s
GBM_2	GBM	0.9912	0.1961	0.9919	35 s
GLM_1	GLM	0.9878	0.1588	0.9893	519 s

B. Tuned hyperparameters

Table 28. Best configuration of each tuner, refit on all of train (models/lgbm_*_params.json; LightGBM names num_leaves, learning_rate, min_child_samples, feature_fraction, bagging_fraction, lambda_l2).

tuner	leaves	learning rate	min child samples	feature fraction	bagging fraction	lambda l2	trees
LightGBM grid	511	0.1	100	0.5	1	0	719
LightGBM random	46	0.04825	72	0.7463	0.9436	0.0001778	739
LightGBM tuned (Optuna)	67	0.09236	7	0.7232	0.9976	4.713e-07	609

C. Experiment log

Table 29. Records in reports/experiments.jsonl by step (every tuning trial, AutoML run, leakage run and evaluation job).

step	records
tuning	123
automl	3
leakage	13
explain_shap	1
eval	16
explain_oof	1
threshold	2

D. Where to find everything

- Write-ups: reports/tuning_writeup.md, automl_comparison.md, leakage.md, temporal_bias.md, explainability.md, critical_eval.md, limitations.md.
- Tables: reports/tables/*.csv; figures: reports/figures/*.png; leaderboards: reports/leaderboards/.
- Models: models/ (Step 2 LightGBM files, AutoML folders), models/final/ (deployed), models/threshold.json.
- App: app/api.py, app/streamlit_app.py; MODEL_CARD.md; DEMO.md.